Comparative Study of Deep Learning Architectures for Automated Waste Classification

Final report on
Deep Learning Project
[ICT-4442]

Submitted By

Krish Uppal (230953478)
Akanshya Chakraborty (230953042)
Arushi Pandey (230911146)
Varun Bramhankar (230953538)



MANIPAL
ACADEMY *of* HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

SCHOOL OF COMPUTER ENGINEERING
MANIPAL INSTITUTE OF TECHNOLOGY,
MANIPAL ACADEMY OF HIGHER EDUCATION

November 2026

Comparative Study of Deep Learning Architectures for Automated Waste Classification

Team 06: Krish Uppal (230953478), Akanshya Chakraborty (230953042), Arushi Pandey (230911146), Varun Bramhankar (230953538)

School of Computer Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, India

https://github.com/Akanshya1595/DL_Project

***Abstract*: Sorting waste into the right stream at the point of disposal depends on recognising what an item is, and camera-based classifiers are a common way to automate that step. This project compares four deep learning architectures on the Kaggle Garbage Classification dataset, which holds 15,515 photographs in 12 classes: a multilayer perceptron on flattened 64×64 pixels, a five-block convolutional network trained from scratch, an ImageNet-pretrained ResNet-50 and an ImageNet-pretrained ViT-B/16. All four share one stratified 70/15/15 split, one preprocessing pipeline and one evaluation protocol, and class imbalance is handled with class-weighted cross-entropy. On the held-out test set ResNet-50 reached the highest macro-F1 (0.978) and an accuracy of 98.0%, followed by ViT-B/16 (0.972), the scratch CNN (0.851) and the MLP (0.604). ResNet-50 has 23.5M parameters and needs 8.3 ms per image at batch size 1, against 85.8M and 12.4 ms for ViT-B/16. Regrouping predictions into four disposal streams raises the accuracy of ResNet-50 to 99.6%. Removing augmentation changes the macro-F1 of the scratch CNN by +0.008. The most frequent error of the best model is clothes predicted as shoes (13 images).**

***Keywords*: waste classification, deep learning, transfer learning, Vision Transformer, convolutional neural network, ResNet-50, class imbalance, image classification**

I. Introduction

Waste that is separated at the point of disposal is easier to recycle than waste pulled out of a mixed stream later. Manual separation is slow and varies from person to person, so camera-based classifiers have been proposed for segregation units and smart bins. Early public work used small photographic collections such as TrashNet, which has about 2,500 images in six classes [1]. Larger collections now exist, and the Kaggle Garbage Classification dataset [2] used here has 15,515 images in 12 classes.

Two properties of waste images make the task harder than a standard object benchmark. The classes are imbalanced: clothes has 5,325 images and brown-glass has 607. Some classes also differ in small ways. Brown, green and white glass differ mainly in colour, and paper and cardboard share a similar texture. A classifier therefore has to be judged on per-class results as well as overall accuracy.

The choice of architecture is a second open question. A multilayer perceptron needs no convolutional machinery. A convolutional network trained from scratch needs only the project data. Pretrained networks bring features learned on ImageNet [3] but cost more memory and time, and a Vision Transformer [4] makes different assumptions about image structure than a residual CNN [5]. A segregation unit has limited hardware, so a useful comparison reports parameters, training time and latency next to accuracy.

This mini project compares one model from each of four families on a single frozen split and reports the full set of metrics for each. The scope is single-item photographs in 12 classes. Each model is trained once with a fixed recipe and one random seed, no model gets a hyperparameter search, and no image comes from a real conveyor belt. These limits are discussed with the results.

II. Literature Review

Dosovitskiy et al. [4] cut an image into 16×16 patches and pass them to a standard Transformer encoder. With large-scale pre-training on ImageNet-21k and JFT-300M the model reached 88.55% top-1 accuracy on ImageNet. That recipe needs very large corpora, and Touvron et al. [6] showed that a ViT trained on ImageNet-1k alone, with heavy augmentation and a distillation token, can reach up to 85.2% top-1. Steiner et al. [7] studied data, augmentation and regularisation for ViTs and found that stronger augmentation and regularisation can make up for a smaller training set, which guides our fine-tuning on about 15,000 images. Liu et al. [8] introduced the Swin Transformer, which uses shifted-window attention and hierarchical features; Swin-L reached 87.3% top-1 with ImageNet-22k pre-training. We kept Swin-Tiny as a fallback in case ViT-B/16 proved too costly to fine-tune.

Convolutional networks have kept pace. ConvNeXt [9] updates a ResNet with Transformer-era design choices; ConvNeXt-T reaches 82.1% top-1 and the larger variants reach up to 87.8% with ImageNet-22k pre-training. Wightman et al. [10] raised ResNet-50 to 80.4% top-1 with an improved training recipe alone, and Bello et al. [11] matched EfficientNet accuracy with ResNets that ran 1.7 to 2.7 times faster on TPUs by changing training and scaling strategies. Both results suggest that the training recipe matters at least as much as the architecture, so we report the full recipe for every model.

Cost matters for a unit that runs in the field. Tan and Le [12] report 87.3% top-1 for EfficientNetV2 with ImageNet-21k pre-training and training up to 11 times faster than earlier models, and we follow their habit of reporting training time next to accuracy. Mehta and Rastegari [13] built MobileViT, a hybrid of convolution and Transformer blocks that reaches 78.4% top-1 on ImageNet-1k with about 6M parameters, which makes it a useful size and latency reference for a classifier that must run inside a segregation unit.

Waste-specific work is thinner. Mao et al. [14] tuned DenseNet121 with a genetic algorithm and report about 99.6% test accuracy on TrashNet, evidence that fine-tuned CNNs do well on controlled images. Most of the waste studies we found report a single fine-tuned CNN on TrashNet under controlled lighting. We did not find a study that compares a fully connected baseline, a scratch CNN, a fine-tuned CNN and a Vision Transformer on one split of a larger 12-class waste dataset, which is the comparison made here.

III. Objectives

1) To compare a multilayer perceptron, a scratch CNN, ResNet-50 and ViT-B/16 on one stratified 70/15/15 split of the 12-class dataset, using test accuracy and macro-F1 as the primary measures, so that differences come from the architecture alone.

2) To report the cost of each model next to its accuracy: parameter count, training time and per-image inference latency.

3) To identify which classes are confused most often, and to measure how much of that error disappears when 12-class predictions are regrouped into four disposal streams.

IV. Methodology

All four models are trained with AdamW [15] on class-weighted cross-entropy, with a linear warm-up over the first 5% of steps followed by cosine decay of the learning rate. Mixed precision is used on the GPU, and the checkpoint with the highest validation macro-F1 is kept. The models are written in PyTorch [16]. Table I gives the configuration of each model.

Table I
Training configuration of the four models

Model	Input	Peak LR	Weight decay	Epochs	Batch	Params (M)
MLP	64×64, flattened	1e-3	5e-4	40	128	12.9
Scratch CNN	128×128	2e-3	5e-4	30	64	2.4
ResNet-50	224×224	1e-3 (head), 1e-4 / 5e-4	1e-4	3 + 7	64	23.5
ViT-B/16	224×224	3e-5 (body), 3e-4 (head)	0.05	5	32	85.8

A. Model 1: multilayer perceptron

The image is resized to 64×64, scaled to [0, 1] and flattened into 12,288 values. Three dense layers follow (12,288 to 1,024 to 256 to 12). The two hidden layers use ReLU, batch normalisation [17] and dropout with probability 0.4 [18]. The model has 12.9M parameters and trains for 40 epochs at a peak learning rate of 1e-3 and a batch size of 128. It is the baseline for the comparison: it has no knowledge of image structure, so any gain of the convolutional models over it shows what spatial structure and pretraining add.

B. Model 2: CNN from scratch

The network has five VGG-style [19] blocks. Each block has two 3×3 convolutions with batch normalisation and ReLU, followed by 2×2 max pooling, and the widths are 32, 64, 128, 256 and 256. Global average pooling, dropout of 0.4 and a linear layer produce the 12 outputs. The input is 128×128 in [0, 1] and the weights are randomly initialised. The model has 2.4M parameters and trains for 30 epochs at a peak learning rate of 2e-3. It shows what convolution and weight sharing give on this dataset when no pretrained features are available. A second run of the same network without augmentation is used for the ablation.

C. Model 3: ResNet-50 with transfer learning

ResNet-50 [5] starts from the ImageNet weights shipped with torchvision, and its final layer is replaced by dropout (0.2) and a 12-way linear layer. Inputs are 224×224 and normalised with the ImageNet mean and standard deviation. Training has two stages. In the first, the backbone is frozen, with batch-normalisation statistics fixed, and only the new head is trained for 3 epochs at a learning rate of 1e-3. In the second, the last two residual stages (layer3 and layer4) and the head are trained for 7 epochs at 1e-4 for the backbone layers and 5e-4 for the head. ResNet-50 is the standard strong CNN for transfer learning, and the literature reviewed above [10], [11] shows that it remains a competitive reference when it is trained well.

D. Model 4: ViT-B/16

The model is the torchvision ViT-B/16 with ImageNet-1k weights and a new 12-way linear head. It splits a 224×224 image into 196 patches of 16×16, adds a class token and passes the sequence through 12 Transformer encoder layers with a hidden size of 768 and 12 attention heads. All weights are fine-tuned for 5 epochs at a learning rate of 3e-5 for the body and 3e-4 for the head, with weight decay of 0.05, gradient clipping at 1.0 and a batch size of 32. This is the attention-based member of the comparison. DeiT [6] and the study of Steiner et al. [7] suggest that a ViT can be fine-tuned on a

dataset of this size when augmentation is used, and the comparison with ResNet-50 shows whether global attention pays off on waste photographs.

V. Experimental Setup

A. Dataset

The data are the Kaggle Garbage Classification collection [2]: 15,515 RGB photographs of single waste items in 12 classes (battery, biological, brown-glass, cardboard, clothes, green-glass, metal, paper, plastic, shoes, trash, white-glass). Every file was opened once to catch corrupt images, and 0 unreadable files were dropped. Fig. 1 shows the class counts; clothes is 8.8 times larger than brown-glass.

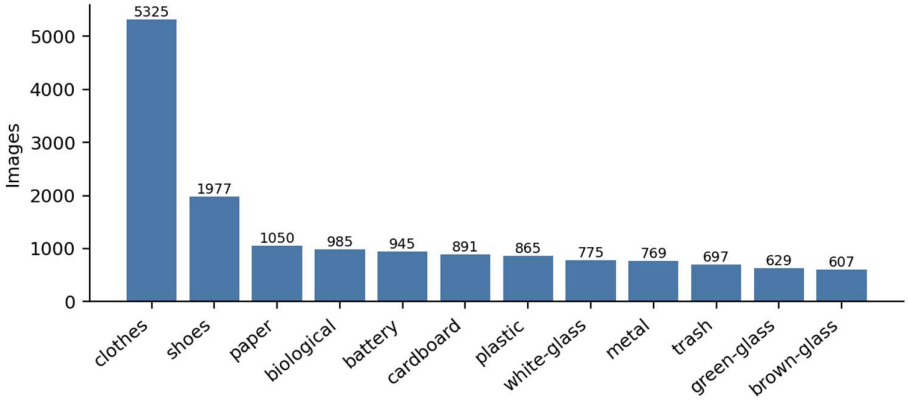


Fig. 1. Number of images per class in the full dataset.

B. Preprocessing

Class names are sorted alphabetically and mapped to integer labels 0 to 11, and the mapping is stored with the code. All images are first resized to 224×224 without cropping. The MLP and CNN receive 64×64 and 128×128 versions of that image with pixels scaled to [0, 1]; ResNet-50 and ViT-B/16 receive 224×224 images normalised with the ImageNet mean and standard deviation. Augmentation is applied to the training set only and runs on the GPU: random horizontal flip, rotation up to ±15°, brightness, contrast and saturation factors drawn from 0.8 to 1.2, and a hue shift of at most ±0.03 of the colour wheel (about 11°). The hue shift is kept small because the three glass classes differ mainly in colour. Validation and test images are only resized and normalised. The loss weights each class by $N/(12 \cdot n_c)$, where N is the training-set size and n_c the number of training images in class c.

C. Train, validation and test split

A stratified 70/15/15 split was generated once with seed 42, which gives 10,860 training, 2,327 validation and 2,328 test images (Table II). The split is saved as a CSV file and every model reads the same indices. The validation set selects the checkpoint, and the test set is used once per model for the final numbers.

TABLE II
CLASS COUNTS AND STRATIFIED SPLIT

Class	Disposal stream	Total	Train	Val	Test
battery	Hazardous	945	661	142	142
biological	Organic	985	689	148	148
brown-glass	Recyclable	607	425	91	91
cardboard	Recyclable	891	624	133	134
clothes	Residual/textile	5,325	3,727	799	799
green-glass	Recyclable	629	440	95	94
metal	Recyclable	769	538	116	115
paper	Recyclable	1,050	735	157	158
plastic	Recyclable	865	606	129	130
shoes	Residual/textile	1,977	1,384	296	297
trash	Residual/textile	697	488	105	104
white-glass	Recyclable	775	543	116	116
Total		**15,515**	**10,860**	**2,327**	**2,328**

A random split of photographs can place near-identical images on both sides. To check this, each image was reduced to a 64-bit difference hash, and a test image was counted as a near-duplicate if some training image lay within a Hamming distance of 4. This applies to 205 of 2,328 test images (8.8%); Section VI-G reports how it affects the scores.



Fig. 2. One example image per class.

D. Evaluation protocol

All models are scored by the same module on the same test images. It reports accuracy, macro-averaged precision, recall and F1 (macro averages weight every class equally, which suits the imbalance), per-class F1, the confusion matrix, the parameter count, wall-clock training time including per-epoch validation, and inference latency. Latency is the mean time of a forward pass over 100 test images at batch size 1 after 20 warm-up passes, measured on Tesla T4. Macro-F1 is the headline metric. Differences between models are checked with McNemar's test on the images where two models disagree, and the 95% normal-approximation interval of an accuracy at 2,328 images is also given. Predictions are regrouped into four disposal streams: recyclable (brown-glass, cardboard, green-glass, metal, paper, plastic, white-glass), organic (biological), hazardous (battery) and residual or textile (clothes, shoes, trash). This grouping is ours and is meant to show how errors look at the level at which a segregation unit would act. All experiments ran on Google Colab with PyTorch 2.11.0+cu130.

VI. Results and Discussion

A. Comparison across models

TABLE III
TEST-SET RESULTS ON THE COMMON PROTOCOL (SINGLE RUN, SEED 42)

Model	Acc. (%)	Macro-P	Macro-R	Macro-F1	Params (M)	Train (min)	Latency (ms)
MLP	67.31	0.588	0.632	0.604	12.9	3.1	0.67
Scratch CNN	87.37	0.843	0.863	0.851	2.4	5.9	1.87
ResNet-50	**98.02**	**0.977**	**0.978**	**0.978**	**23.5**	**4.3**	**8.27**
ViT-B/16	97.72	0.971	0.974	0.972	85.8	11.2	12.39

Table III lists the test results; the best row is in bold. ResNet-50 scored highest with a macro-F1 of 0.978 and an accuracy of 98.0%. ViT-B/16 followed at 0.972, then the scratch CNN at 0.851 and the MLP at 0.604. At 2,328 test images the 95% interval around a single accuracy is about ±0.6 points for ResNet-50. The gap between ResNet-50 and ViT-B/16 is too small to separate in a single run: McNemar's test on the images where they disagree (25 right only for ResNet-50, 18 only for ViT-B/16) gives $p = 0.36$. Test macro-F1 stays within 0.027 of the validation value for every model, so selecting checkpoints on the validation set did not overfit it.

Fig. 3 shows how validation macro-F1 and loss developed during training.

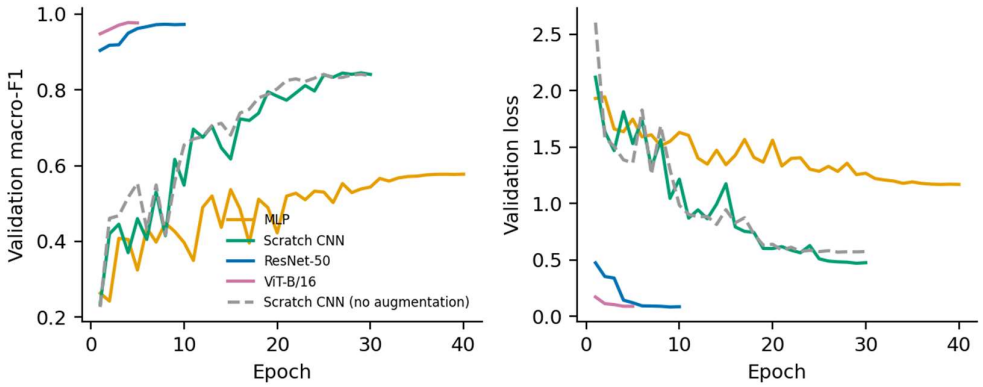


Fig. 3. Validation macro-F1 and validation loss per epoch. The dashed line is the scratch CNN without augmentation.

B. Why the models differ

The MLP has the lowest macro-F1 (0.604). It treats every pixel as an independent input, so an object that is shifted, rotated or flipped looks like a different pattern, and its first layer alone holds 12.6 million weights for about 10,860 training images.

The scratch CNN improves on the MLP by 24.7 macro-F1 points while using fewer parameters (2.4M against 12.9M), so the gain cannot come from extra capacity. Shared convolutional filters respond to the same pattern wherever it appears in the image, and pooling adds tolerance to small shifts, which the flattened MLP lacks.

Both pretrained networks beat the scratch CNN, ResNet-50 by 12.6 points and ViT-B/16 by 12.1. About 10,860 images are few for learning visual features from random initialisation. ImageNet pretraining [3] already supplies edge, texture and shape detectors, and the fine-tuning stage only has to adapt them to waste photographs and learn the new classifier.

ResNet-50 finished 0.6 macro-F1 points ahead of ViT-B/16. A plausible reason is that the convolutional prior (local filters, weight sharing) suits a dataset of about 10,860 images, while a ViT has weaker built-in assumptions and may need more data or more epochs than the 5 used here to make use of its extra capacity. We did not run an experiment that isolates this explanation.

Fig. 4 gives the per-class F1 of every model.

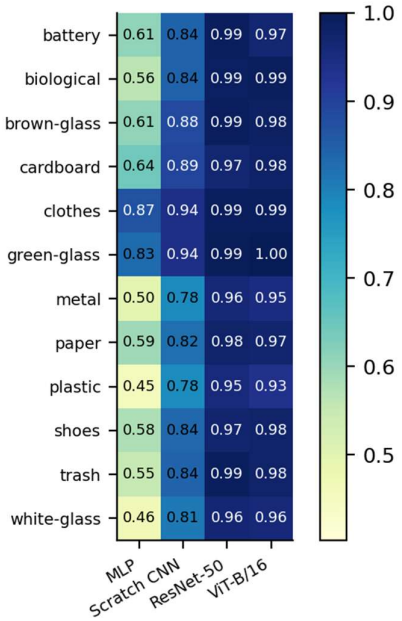


Fig. 4. Per-class F1 on the test set for each model.

C. Cost and deployment

ViT-B/16 has 85.8M parameters, 3.6 times the 23.5M of ResNet-50, and its latency is 12.4 ms per image against 8.3 ms. Training took 3.1 min for the MLP, 5.9 min for the scratch CNN, 4.3 min for ResNet-50 and 11.2 min for ViT-B/16. On a GPU at batch size 1, the MLP and CNN latencies are dominated by kernel-launch overhead, so the latency ratios understate the difference in arithmetic cost. Fig. 5 plots macro-F1 against latency. ResNet-50 is the most accurate model, and whether its cost is acceptable depends on the hardware of the segregation unit.

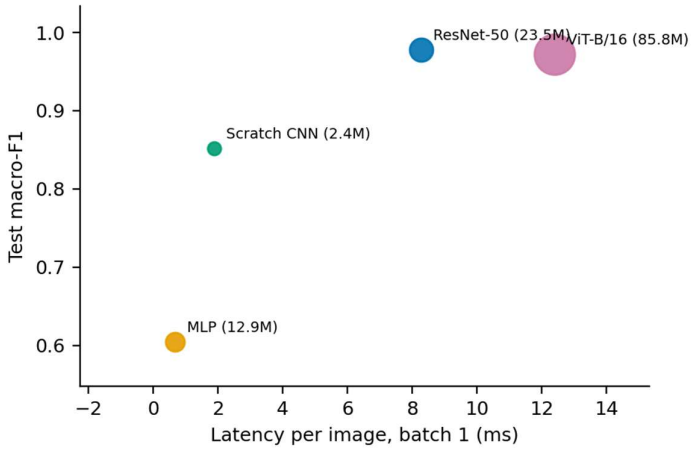


Fig. 5. Test macro-F1 against latency at batch size 1. Marker size grows with parameter count.

D. Error analysis of the best model

ResNet-50 made 46 errors on 2,328 test images (2.0%). Its lowest per-class F1 scores are plastic (0.954), metal (0.961) and white-glass (0.965), and its highest are green-glass (0.995), trash (0.990) and battery (0.989). Table IV lists the most frequent confusions.

TABLE IV
MOST FREQUENT CONFUSIONS OF RESNET-50 ON THE TEST SET

True class	Predicted class	Images	Same disposal stream
clothes	shoes	13	yes
white-glass	plastic	5	yes
shoes	clothes	4	yes
cardboard	paper	4	yes
biological	cardboard	3	no
plastic	metal	2	yes

37 of the 46 errors (80.4%) are between two classes in the same disposal stream and would not change where the item ends up. Paper and cardboard are also confused, which matches their similar texture. Fig. 6 shows the confusion matrix, and Fig. 7 shows the 12 test images that the model got wrong with the highest confidence.

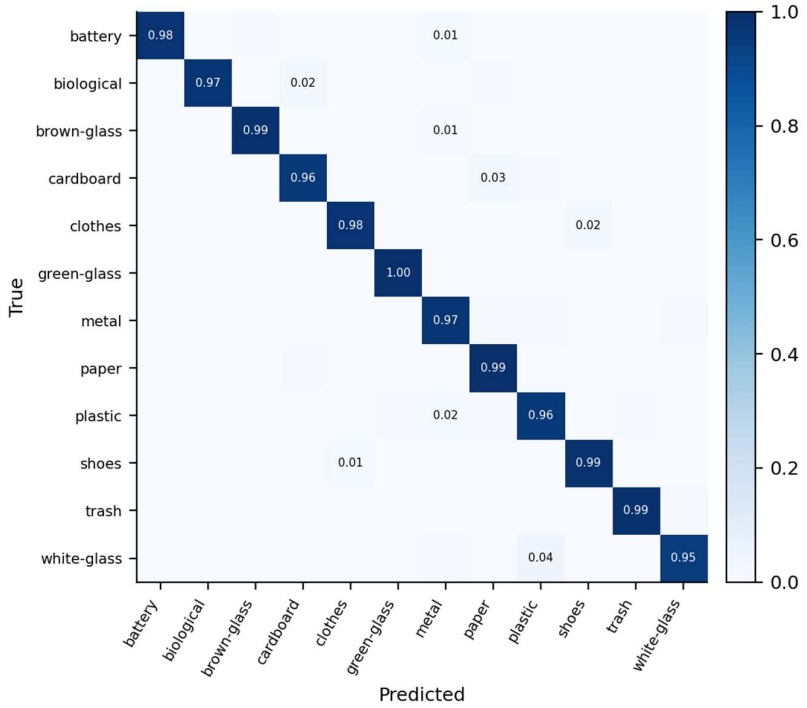


Fig. 6. Row-normalised confusion matrix of ResNet-50 on the test set.



Fig. 7. The test images ResNet-50 misclassified with the highest confidence (softmax probability in brackets).

The model is less confident when it is wrong: the mean softmax probability is 0.80 for errors and 0.99 for correct predictions. If only predictions with confidence of at least 0.9 are accepted, 96.7% of test images are kept at an accuracy of 99.1%. A unit could send the remaining images to a second sensor or a human.

E. Disposal-stream view

TABLE V
ACCURACY AT THE 12-CLASS LEVEL AND AFTER REGROUPING INTO FOUR DISPOSAL STREAMS

Model	12-class acc.	4-stream acc.	4-stream macro-F1	Errors removed
MLP	67.3%	78.8%	0.700	35%
Scratch CNN	87.4%	92.3%	0.887	39%
ResNet-50	**98.0%**	**99.6%**	**0.992**	**80%**
ViT-B/16	97.7%	99.3%	0.986	70%

Table V shows that regrouping raises the accuracy of ResNet-50 from 98.0% to 99.6%, which removes 80% of its errors. The gain from regrouping is 11.5 points for the MLP, 4.9 points for the scratch CNN, 1.6 points for ResNet-50 and 1.6 points for ViT-B/16.

F. Effect of augmentation

Table VI compares the scratch CNN trained with and without the augmentation pipeline, with the same seed, split and schedule (Fig. 8).

TABLE VI
SCRATCH CNN WITH AND WITHOUT AUGMENTATION (TEST SET)

Setting	Acc. (%)	Macro-F1	Best epoch	Best val. macro-F1
With augmentation	87.37	0.851	29	0.845
Without augmentation	88.02	0.859	29	0.841

Augmentation lowered the macro-F1 of the scratch CNN by 0.008. A likely reason is that 30 epochs is short for a network learning from random initialisation under augmentation, or that rotation and colour changes remove cues the classes rely on. We did not run further experiments to separate these causes.

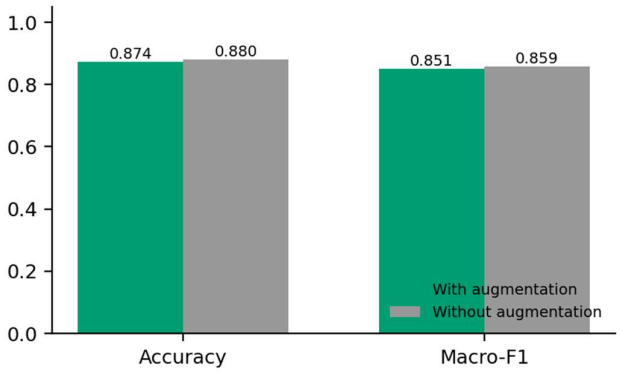


Fig. 8. Scratch CNN test accuracy and macro-F1 with and without augmentation.

G. Near-duplicate check

205 test images (8.8%) have a near-duplicate in the training set. ResNet-50 scores 98.5% on those images and 98.0% on the other 2,123. The higher figure on near-duplicates suggests the full test accuracy is slightly flattered, and the score on the remaining images is the more conservative one. On the de-duplicated subset the accuracies are MLP 66.8%, Scratch CNN 86.9%, ResNet-50 98.0% and ViT-B/16 97.7%.

VII. Conclusion

On this dataset and split, ResNet-50 performed best, with a test macro-F1 of 0.978 and an accuracy of 98.0%. The ranking by macro-F1 was ResNet-50 (0.978), ViT-B/16 (0.972), the scratch CNN (0.851) and the MLP (0.604). Pretraining separated the top two models from the scratch CNN and the MLP, and the gain of the convolutional scratch model over the MLP came with fewer parameters. The difference between ResNet-50 and ViT-B/16 is not statistically significant in our single run (McNemar $p = 0.36$), so the cheaper of the two may be the better engineering choice. The most frequent confusion of the best model is clothes predicted as shoes, 37 of its 46 errors stay inside one disposal stream, and regrouping into four streams raises its accuracy to 99.6%. Augmentation changed the macro-F1 of the scratch CNN by -0.008.

VIII. Future Work

The results come with limits that suggest the next steps. Each model was trained once with one seed and a fixed recipe, so repeated runs with several seeds, reported as means with intervals, would show how much of the gaps between close models is noise, and a proper hyperparameter search for every model would make the comparison fairer. The optional cross-dataset check on TrashNet, planned in the interim report, was not carried out; it would test whether the scores hold on images taken under different conditions. The dataset contains single items photographed one at a time, so the next test should use images from a real conveyor belt with clutter, occlusion and varying light.

Other architectures from the literature review could be added to the comparison, namely Swin Transformer [8], ConvNeXt [9] and the lightweight MobileViT [13], the last of which targets exactly the size and latency limits of an embedded unit. Compression by quantisation or distillation could bring the larger models within those limits. The confidence analysis in Section VI-D suggests a rejection rule that sends uncertain items to a human or a second sensor, and a hierarchical classifier that first decides the disposal stream and then the class would match the way the output is used. Finally, the stream grouping used here is our own, and a grouping agreed with a waste-management partner would make the stream-level results more meaningful.

Acknowledgement

The authors thank the faculty of the School of Computer Engineering, Manipal Institute of Technology, for guidance during the Deep Learning course project (ICT-4442). Training was run on free GPU resources from Google Colab, and the dataset was obtained from Kaggle.

References

[1] M. Yang and G. Thung, "Classification of trash for recyclability status," CS229 Project Report, Stanford Univ., 2016.

[2] M. Abla, "Garbage classification (12 classes)," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/mostafaabla/garbage-classification

[3] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2009, pp. 248-255.

[4] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learn. Represent. (ICLR), 2021.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770-778.

[6] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers & distillation through attention," in Proc. Int. Conf. Mach. Learn. (ICML), 2021, pp. 10347-10357.

[7] A. Steiner, A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit, and L. Beyer, "How to train your ViT? Data, augmentation, and regularization in vision transformers," Trans. Mach. Learn. Res., 2022.

[8] Z. Liu et al., "Swin Transformer: Hierarchical vision transformer using shifted windows," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2021, pp. 10012-10022.

[9] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 11976-11986.

[10] R. Wightman, H. Touvron, and H. Jegou, "ResNet strikes back: An improved training procedure in timm," arXiv:2110.00476, 2021.

[11] I. Bello et al., "Revisiting ResNets: Improved training and scaling strategies," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 34, 2021.

[12] M. Tan and Q. Le, "EfficientNetV2: Smaller models and faster training," in Proc. Int. Conf. Mach. Learn. (ICML), 2021, pp. 10096-10106.

[13] S. Mehta and M. Rastegari, "MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer," in Proc. Int. Conf. Learn. Represent. (ICLR), 2022.

[14] W.-L. Mao, W.-C. Chen, C.-T. Wang, and Y.-H. Lin, "Recycling waste classification using optimized convolutional neural network," Resour. Conserv. Recycl., vol. 164, Art. no. 105132, 2021.

[15] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019.

[16] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 32, 2019.

[17] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. Int. Conf. Mach. Learn. (ICML), 2015, pp. 448-456.

[18] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, no. 56, pp. 1929-1958, 2014.

[19] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in Proc. Int. Conf. Learn. Represent. (ICLR), 2015.

C.3: Individual Contribution Statement

Member name	Reg. No.	Model / task owned	Key contributions	Signature
Krish Uppal	230953478	Model 3: ResNet-50; shared evaluation module	Created the GitHub repository with one branch per member. Wrote the shared evaluation module (accuracy, macro precision, recall and F1, per-class F1, confusion matrix, parameter count, training time, latency). Implemented, trained and tuned ResNet-50. Wrote the dataset and models sections.	Krish Uppal
Akanshya Chakraborty	230953042	Model 4: ViT-B/16; comparison table	Built the ViT-B/16 data pipeline and fine-tuning script and trained the model. Prepared the single comparison table for all four models. Wrote the literature review rows for the Transformer papers and the risk and plan section.	Akanshya Chakraborty
Arushi Pandey	230911146	Model 2: CNN from scratch; augmentation ablation	Implemented and trained the scratch CNN and ran the with/without augmentation ablation. Wrote the literature review rows for the convolutional papers.	Arushi Pandey
Varun Bramhankar	230953538	Model 1: MLP; dataset and split	Downloaded the dataset, carried out the exploratory analysis, and generated and froze the stratified train/validation/test split. Implemented and trained the MLP. Wrote the preprocessing section.	Varun Bramhankar